Fundamentals of 3D Perspective Projection In Computer Graphics

Saehim Park

December 22, 2025

Abstract

A view of the world with human eyes, and approximations of 3D spaces on computer screens are both 2-dimensional. In both cases, the interaction of light with objects, and the subsequent projection of that light onto a surface is what makes up the 2D view of a 3D space. This paper will seek to computationally and programmatically implement and simulate these real world physical interactions, in order to help readers gain a deeper and fuller understanding of the techniques that underlie modern 3D computer graphics, specifically ray tracing and rasterization. A full description of the concepts and implementation process will be provided, with the aim of providing readers with intuitive understanding of the methods used.

Contents

1 Introduction **3**

2 3 Dimensional Space **4**
2.1 Coordinates and Vectors 4

3 Ray Tracing **9**
3.1 Viewports and Rays 9
3.2 A Ray Tracer Implementation 10
3.3 Camera Movement 18

4 Rasterization **23**
4.1 Polygon Projection 23
4.2 Model Transformation 30

5 Conclusion **39**

List of Figures

1 The point (1, 1, 1) in a 3D cartesian coordinate system 4
2 Right-hand rule 5
3 A vector from point $(0, 0, 0)$ to $(1, 1, 1)$ 6
4 The vector $\langle 1, 2, 3 \rangle$ broken down into its x, y, and z components 6
5 7

6 The cross product of two vectors $\langle 0, 1, 1\rangle$ and $\langle 1, 1, 1\rangle$ 8
7 A viewport of a 16:9 screen with its center located at $(0, 0, 1)$ . . 9
8 A vector from camera to pixel $(0, 0)$'s equivalent location on viewport 10
9 Coordinate ranges for pixels on a $1920 \cdot 1080$ screen 14
10 15
11 A 3D model approximated into triangles 23
12 A demonstration of polygon triangulation 23
13 A side (yz plane) view of an arbitrary point P 24
14 Projection of a 3D triangle onto a 2D viewport 25
15 Disconnected lines 26
16 29
17 Our filled triangle result 30
18 The vector $2\hat{\imath} + 1\hat{\jmath}$ 32
19 The angle between $\vec{V}$ and $\vec{V\prime}$ is equal to the angle between $\hat{\imath}$ and $\hat{\imath\prime}$ 33
20 A preview of the rotation mechanism in Blender 37
21 A model of a tree rendered with our rasterizer 39

1 Introduction

The world is 3-dimensional. In contrast, visual perceptions of the world are 2-dimensional, and yet, recognizable as of a 3D space. Additionally, through this view, one can still perceive the depth of the image, retrieving some of the information lost in the process of flattening the view into 2 dimensions. This phenomenon reveals the possibility of projecting a 3D space onto a 2D plane, allowing us to emulate the existence of a 3D space behind a 2D plane (much like looking through a window to look at the outside world; a detailed enough 2D image on the window would be indistinguishable from a view of the outside).

Furthermore, this possibility is taken advantage of to form the foundations of modern computer graphics. When it comes to video games, computer generated imagery, or 3D animation, computers must simulate all the physical processes of the real world that produce the light and resulting image that reaches your eyes. Although the entirety of the physical world cannot possibly be simulated within the constraints of modern computing capabilities, mathematical and physical abstractions can be made in order to simplify this process and produce near true to life results. These abstractions form the basis of 3D computer graphics.

In modern computing, it is unnecessary to know the fundamental concepts and processes behind projecting a 3D image; with the release of many software solutions and libraries that aid in the development of appropriate visuals, the modern day developer does not need to grasp the underlying mechanics behind 3D computer graphics in order to produce satisfactory results. However, the benefits of learning the fundamental process behind this widespread field are essential for those who wish to become experts not tied down by any specific engine or library. Factors such as performance, scalability, and customizability justify the process of learning the fundamentals. Above all else, those who understand are better equipped to take advantage of and develop graphical toolkits. As such, it is imperative to understand the foundations that make up 3D computer graphics for those who wish to become true masters of the field.

This paper sets out to document the process of learning the fundamentals of 3D computer graphics, with real code examples mapping out the learning process. As such, it aims to serve as a useful guide that directs readers to explore the bare-bones fundamentals of 3D computer graphics.

2 3 Dimensional Space

2.1 Coordinates and Vectors

In order to represent a 3-dimensional space, we must first develop the tools used for defining a 3D space.

A **cartesian coordinate system** is a system of numerically representing space with a set of coordinates. n-dimension spaces require n sets of coordinates to represent. As such, a 3-dimension cartesian coordinate system requires 3 sets of coordinates, commonly referred to as x, y, and z.

A **point** is a single position in space, represented by a set of coordinates, and notated by $(C_1, C_2, C_3, \ldots, C_n)$. A 3D cartesian coordinate system places the center, or the origin point at $(0, 0, 0)$. Each dimension, represented through an axis, has a direction: traveling towards the direction of an axis is known as traveling positively, while traveling against the direction of an axis is known as traveling negatively, with the corresponding coordinates increasing or decreasing appropriately.[1]

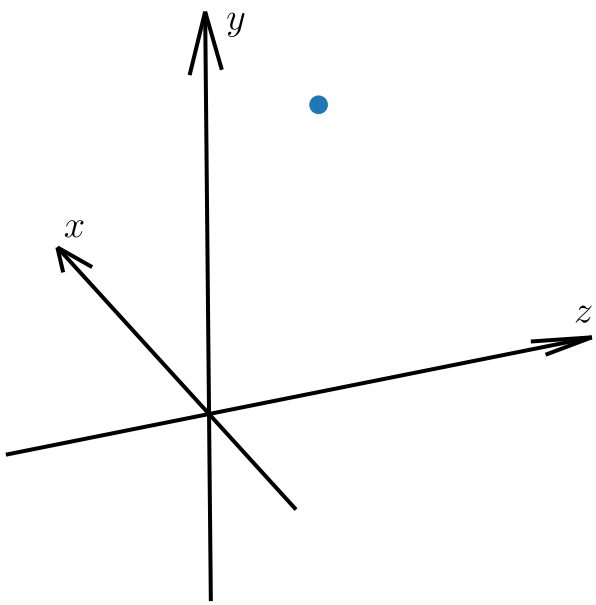


Figure 1: The point (1, 1, 1) in a 3D cartesian coordinate system

The directions of each axis in relation to each other can be remembered using the right hand rule (Figure 2). Axis configurations corresponding to the left hand also exist, however the right-handed system is more common.

[1] Eldridge, *Cartesian coordinates*.

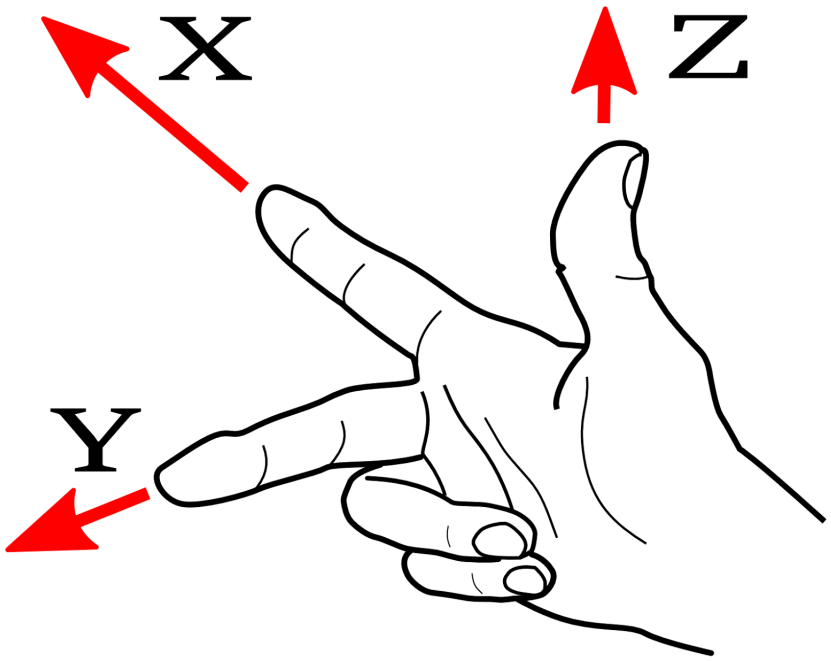


Figure 2: Right-hand rule

Vectors are the differences between points. Vectors can be used to represent many things, such as motion and direction. At its simplest, a vector can be intuitively thought of as a set of components that represent a certain magnitude of motion in a certain direction. It can also be thought of as the concept of a slope used in 2D cartesian coordinate systems expanded for n-dimensions. Visually, it is a line connecting two points in space (Figure 3).

n-dimension vectors can be notated as a set of n components:

$$\vec{V} = \begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ \vdots \\ V_n \end{bmatrix} \quad \text{or} \quad \vec{V} = \langle V_1, V_2, V_3, \ldots, V_n \rangle$$

Much like the slope of a line in 2 dimensions, vectors can be thought of as the ratio between each of the components' deltas (differences in magnitude). For example, take the 3D vector $\langle 1, 2, 3 \rangle$: when x is increased by 1, y is increased by 2, and z is increased by 3 (Figure 4).[2]

Arithmetic operations can also be performed on vectors. Vector addition is done by summing together each respective component. For example, the resulting vector of adding vectors $\langle A_x, A_y, A_z \rangle$ and $\langle B_x, B_y, B_z \rangle$ together is vector $\langle A_x + B_x, A_y + B_y, A_z + B_z \rangle$. Visually, adding two vectors involves putting the beginning of a vector at the end of another (Figure 5). Additionally, two

[2]Encyclopedia Britannica, *vector*.

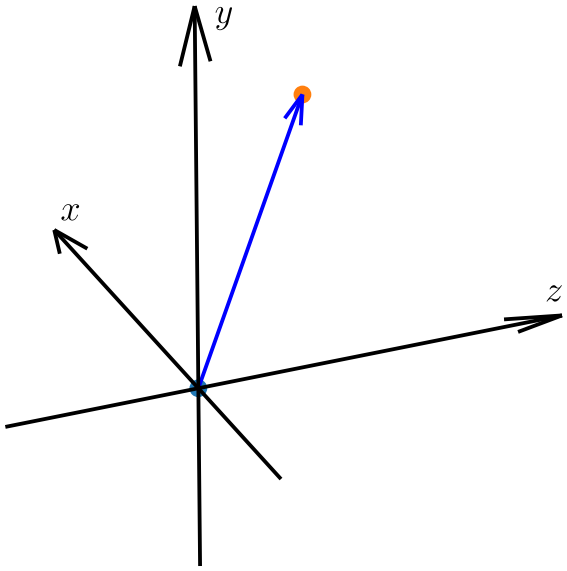


Figure 3: A vector from point $(0, 0, 0)$ to $(1, 1, 1)$

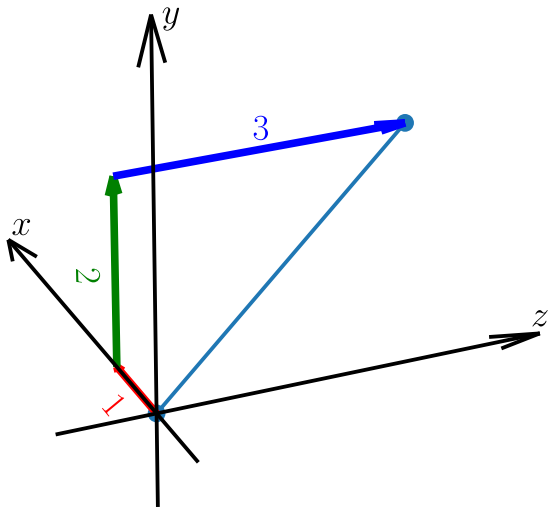


Figure 4: The vector $\langle 1, 2, 3 \rangle$ broken down into its x, y, and z components

types of vector multiplication exist: the **dot product** and the **cross product**. The dot product (denoted as $\vec{A} \cdot \vec{B}$) is used to find how similar two vectors are to each other, and results in a scalar[3] value representing similarity. A vector dot product with itself produces a scalar square of its magnitude. The cross product (denoted as $\vec{A} \times \vec{B}$) by contrast is used to find a vector that is the most different to the vectors it operates on. The cross product of two vectors results in a vector perpendicular to the plane created by the two vectors (Figure 6). Formulas for the dot product (1) and the cross product (2) are listed below.

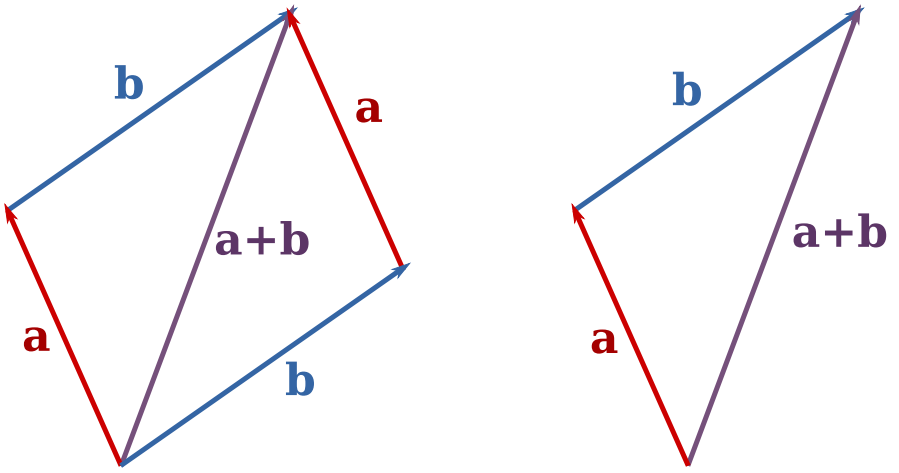


Figure 5

$$\vec{A} \cdot \vec{B} = A_x B_x + A_y B_y + A_z B_z \tag{1}$$

$$\vec{A} \times \vec{B} = \begin{bmatrix} A_y B_z - A_z B_y \\ A_z B_x - A_x B_z \\ A_x B_y - A_y B_x \end{bmatrix} \tag{2}$$

Notably, the dot product is nothing more than the sum of the products of each of the components; as such, the commutative and distributive properties of arithmetic multiplication also apply.

$$\vec{A} \cdot \vec{B} = \vec{B} \cdot \vec{A}$$
$$(\vec{A} + \vec{B}) \cdot \vec{C} = \vec{A} \cdot \vec{C} + \vec{B} \cdot \vec{C}$$

Scalar multiplication of vectors is also possible.

$$t\vec{V} = \langle tV_x, tV_y, tV_z \rangle$$

Vectors are useful in that they allow us to easily define movement or position differences in any direction.

[3] A scalar is a value that possesses only magnitude. It differs from a vector value in that vectors possess both magnitude and direction.

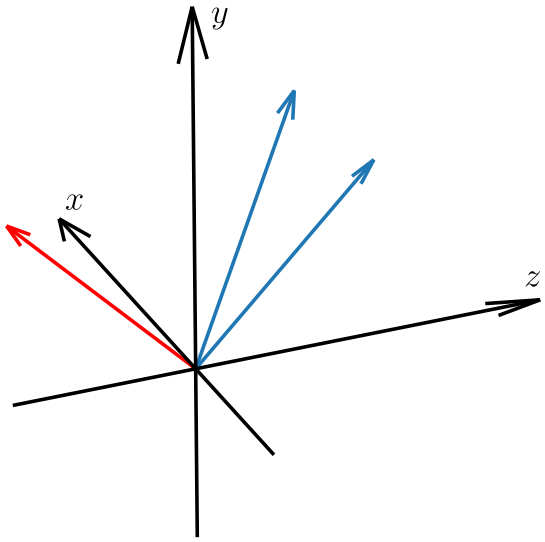


Figure 6: The cross product of two vectors $\langle 0, 1, 1 \rangle$ and $\langle 1, 1, 1 \rangle$

A **line** is a collection of points arranged in a straight path, extending infinitely in opposing directions.[4] In 2D, a line is most commonly defined as an equation conveying the relationship between the two coordinate variables x and y (3). Although this can be expanded to 3 dimensions (3), a more convenient and commonly used definition utilizes vectors (4). In this definition, every point P along the line is defined by adding a multiple t ($t \in \mathbb{R}$) of $\vec{D}$ to a point of origin O. Intuitively, it can be thought of as starting from an origin point O and traveling t amount in the direction of the vector component $\vec{D}$.

Line definitions allow us to calculate intersections involving objects traveling in a straight path.

$$y = mx + b \tag{3}$$

$$\frac{x - x_0}{a} = \frac{y - y_0}{b} = \frac{z - z_0}{c} \tag{4}$$

$$P = O + t\vec{D} \tag{5}$$

Now that we have established a system of rules for representing space numerically, we can begin to lay the groundwork for projecting a 3D space onto a 2D plane.

[4]Encyclopedia Britannica, *line*.

3 Ray Tracing

3.1 Viewports and Rays

The process of projecting a 3D view onto a 2D plane is inspired by how vision and perspective works in the real world. Vision in the real world is essentially a 2D image of a 3D landscape.

A person is looking out onto a landscape and would like to save this view onto a canvas. An artist could simply paint the view, however this person lacks the skills required. Cleverly, they set up a net the same size as the canvas in a position that captures the desired view. Looking through a square hole of the net, the person averages out the color that they can see through that hole and paints the location corresponding to that hole on the canvas with that color. Repeating this step for every single hole results in an accurate sketch of the landscape.[5]

This metaphor is useful in that it can be applied to computers. Computers display images as a grid of square pixels, much like the square holes of the net used in the analogy.

A rectangle referred to as the **viewport** is placed in the world space, the equivalent to the square net. Then a camera is placed behind the viewport, the equivalent to the person's eyes (Figure-7).

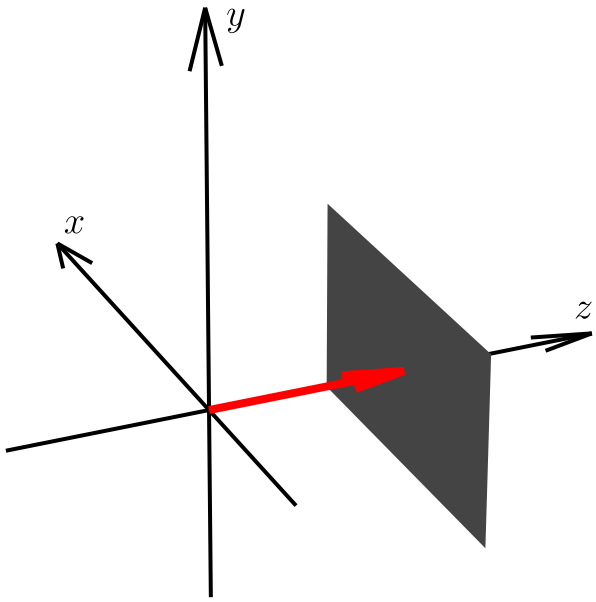


Figure 7: A viewport of a 16:9 screen with its center located at $(0, 0, 1)$

[5]Gambetta, *Computer Graphics From Scratch*.

In the net analogy, the colors that reach the person's eyes are certain wavelengths of photons that travel from an origin light source, bounce off reflective objects, and ultimately end up in their retinas. Although we could simulate photons traveling in all direction from a light source, that would be highly redundant as most of them would not end up hitting the camera. Instead, it is more efficient to trace in reverse: light rays traveling outwards from the camera and into any objects.

A **ray** is a collection of points arranged in a straight path extending infinitely towards a single direction. Whereas a line extends infinitely in two opposing directions from a point, rays only extend in a single direction.

In practice, this means that we compute shooting a light ray from the camera to a pixel's equivalent location on the viewport, and if it intersects with an object, color in that pixel on the screen with the color of the object. We repeat this for every pixel until every pixel on the screen is correctly colored in. This technique is known as **ray tracing**.

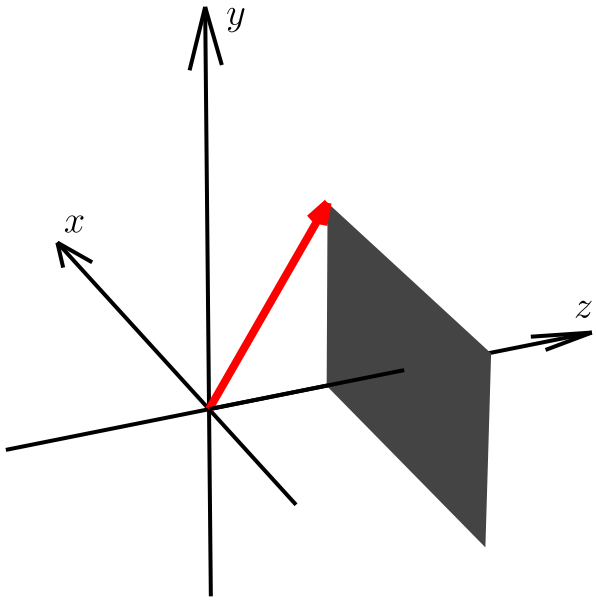


Figure 8: A vector from camera to pixel $(0, 0)$'s equivalent location on viewport

Now that a system for projecting a 3D space onto a 2D plane has been established, what remains is to implement said system.

3.2 A Ray Tracer Implementation

With a ray tracing system defined, an implementation can be programmed. The implementation in this paper will be coded in the C++ language, and the Raylib

graphics library with the raylib-cpp wrapper will be used for drawing pixels onto a window.[6][7]

First, we must define a point, vector, and camera object (C++ class type).

Source Code 1: point.hpp

```
#ifndef POINT_H
#define POINT_H

struct Point {
    double x {};
    double y {};
    double z {};
};

static constexpr Point ORIGIN {0, 0, 0};

#endif
```

Source Code 2: vector.hpp

```
#ifndef VECTOR_H
#define VECTOR_H
#include "point.hpp"

struct Vector {
    double x {};
    double y {};
    double z {};
};

inline constexpr Vector operator+(const Vector& vector1, const
↪  Vector& vector2) {
    return {vector1.x + vector2.x,
            vector1.y + vector2.y,
            vector1.z + vector2.z};
}

inline constexpr Vector& operator+=(Vector& vector1, const
↪  Vector& vector2) {
    vector1 = {
            vector1.x + vector2.x,
            vector1.y + vector2.y,
            vector1.z + vector2.z
        };
    return vector1;
}
```

[6] Santamaria, *Raylib*.
[7] Loach, *raylib-cpp*.

```
inline constexpr Vector operator-(const Vector& vector1, const
↪  Vector& vector2) {
    return {vector1.x - vector2.x,
            vector1.y - vector2.y,
            vector1.z - vector2.z};
}

inline constexpr Vector operator-(const Vector& vector) {
    return {-vector.x,
            -vector.y,
            -vector.z};
}

inline constexpr Vector operator-(const Point& point1, const
↪  Point& point2) {
    return {point1.x - point2.x,
            point1.y - point2.y,
            point1.z - point2.z};
}

inline constexpr Vector operator*(double multiple, const Vector&
↪  vector) {
    return {multiple * vector.x,
            multiple * vector.y,
            multiple * vector.z};
}

inline constexpr double dotProduct(const Vector& vector1, const
↪  Vector& vector2) {
    return vector1.x * vector2.x +
    vector1.y * vector2.y +
    vector1.z * vector2.z;
}

inline constexpr Vector crossProduct(const Vector& vector1, const
↪  Vector& vector2) {
    double a1 = vector1.x;
    double a2 = vector1.y;
    double a3 = vector1.z;

    double b1 = vector2.x;
    double b2 = vector2.y;
    double b3 = vector2.z;

    return Vector{a2 * b3 - a3 * b2, a3 * b1 - a1 * b3, a1 * b2 -
    ↪  a2 * b1};
}

#endif
```

Source Code 3: camera.hpp

```
#ifndef CUSTOMCAMERA_H
#define CUSTOMCAMERA_H
#include "line.hpp"

namespace Custom {
    class Camera {
        public:
            constexpr const Point& pos() const {return m_pos;}

            constexpr double viewportWidth() const {return
            ↪  m_viewportWidth;}
            constexpr double viewportHeight() const {return
            ↪  m_viewportHeight;}

            constexpr int screenWidth() const {return
            ↪  m_screenWidth;}
            constexpr int screenHeight() const {return
            ↪  m_screenHeight;}

        private:
            Point m_pos {0, 0, 0};
            int m_screenWidth {960};
            int m_screenHeight {540};

            double m_viewportDistanceFromCamera {1};
            double m_viewportHeight {1};
            double m_viewportWidth
            ↪  {m_viewportHeight/m_screenHeight *
            ↪  m_screenWidth};
    };
}

#endif
```

Next, we must define a function to convert pixel coordinates to equivalent in-world coordinates on the viewport. For now, we will assume the viewport center to be at $(0, 0, 1)$ and the camera to be at the origin $(0, 0, 0)$ for the sake of simplicity. Pixels on the screen are usually represented with a 2D cartesian coordinate system with an inverted Y-Axis (Figure-9).

For each pixel on the screen, its horizontal (x) distance from the center and vertical (y) distance from the center can be calculated.

```
double xDistanceFromCenter
↪  {camera.viewportWidth()/camera.screenWidth() * -(pixelX -
↪  camera.screenWidth()/2)};
double yDistanceFromCenter
↪  {camera.viewportHeight()/camera.screenHeight() * -(pixelY -
↪  camera.screenHeight()/2)};
```

Adding `xDistanceFromCenter` and `yDistanceFromCenter` to the x and y components of the vector going from camera to viewport center $(0, 0, 1)$ respectively



Figure 9: Coordinate ranges for pixels on a $1920 \cdot 1080$ screen

results in a vector from the camera to the pixel's equivalent location on the viewport.

```
inline constexpr Line getPixelLine(int pixelX, int pixelY, const
↪  Custom::Camera& camera) {

    double xDistanceFromCenter
    ↪  {camera.viewportWidth()/camera.screenWidth() * -(pixelX -
    ↪  camera.screenWidth()/2)};
    double yDistanceFromCenter
    ↪  {camera.viewportHeight()/camera.screenHeight() * -(pixelY
    ↪  - camera.screenHeight()/2)};

    return {camera.pos(), {xDistanceFromCenter,
    ↪  yDistanceFromCenter, 1}};
} // get vector from origin to pixel's equivalent location on
↪  viewport
```

With this, a ray going starting from the camera position point (C_p) and going in the direction of the camera to pixel vector ($\vec{D}$) can be derived.

$$P = C_p + t\vec{D}(t \geq 0)$$

This is our light ray for the pixel (Figure-10).

Now that a method for obtaining the light ray has been determined, we can begin to define objects to display. For this early implementation, a sphere will be used as it is mathematically simple. A sphere is defined as a collection of points that are the same distance away from a single point. Every point P along the surface of a sphere with center C and radius r can be defined as such:

$$distance(P, C) = r^2$$

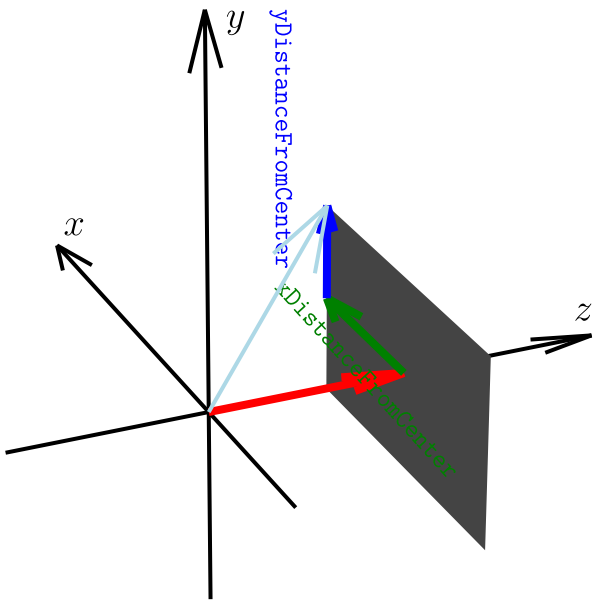


Figure 10

The function $distance(x, y)$ takes in two points as input and outputs the distance between the two points.

All that's left is to determine whether the light ray for our pixel and any point on the sphere intersect; if so, color in that pixel.

By substituting the point on the surface of the sphere P with the equation for our light ray, we can apply the discriminant to the resulting equation in order to determine if a solution t such that the sphere and ray intersect exist.

$$P = C_p + t\vec{D}(t \geq 0)$$
$$distance(P, C) = r^2$$

$$distance(C_p + t\vec{D}, C) = r^2 \tag{6}$$

$$\{(C_p + t\vec{D}) - C\} \cdot \{(C_p + t\vec{D}) - C\} = r^2 \tag{7}$$

$$\{t\vec{D} + (C_p - C)\} \cdot \{t\vec{D} + (C_p - C)\} = r^2 \tag{8}$$

$$t\vec{D} \cdot \{t\vec{D} + (C_p - C)\} + (C_p - C) \cdot \{t\vec{D} + (C_p - C)\} = r^2 \tag{9}$$

$$(t\vec{D} \cdot t\vec{D}) + \{t\vec{D} \cdot (C_p - C)\} + \{t\vec{D} \cdot (C_p - C)\} + \{(C_p - C) \cdot (C_p - C)\} = r^2 \tag{10}$$

$$(\vec{D} \cdot \vec{D})t^2 + 2\{\vec{D} \cdot (C_p - C)\}t + (C_p - C) \cdot (C_p - C) - r^2 = 0 \tag{11}$$

Taking advantage of the fact that the distance between two points is essentially the magnitude of the vector from one point to the other, the *distance* function can then be expanded to the dot product of $\{(C_p + t\vec{D}) - C\}$ with itself (7). Using the distributive property of dot products, the equation can then futher be expanded to (9) and (10). Variable t can be extracted from the dot product to form a quadratic equation (11). To check that a value of t exists such that the equation holds, the discriminant can be applied to the equation.

$$\left[2\{\vec{D} \cdot (C_p - C)\}\right]^2 - 4(\vec{D} \cdot \vec{D})\{(C_p - C) \cdot (C_p - C) - r^2\} \geq 0$$

A function that takes in a light ray and sphere object and outputs whether they intersect can be implemented as such:

```
#ifndef SPHERE_H
#define SPHERE_H
#include <raylib-cpp.hpp>

struct Sphere {
        Point center {0.0, 0.0, 0.0};
        double radius {1};
        raylib::Color color {RED};
};

#endif


inline constexpr bool rayIntersectWithSphere(const Line& ray,
↪   const Sphere& sphere) {
    const auto a {dotProduct(ray.direction, ray.direction)};
    const auto b {2 * dotProduct(ray.origin - sphere.center,
    ↪   ray.direction)};
    const auto c {dotProduct(ray.origin - sphere.center,
    ↪   ray.origin - sphere.center) - pow(sphere.radius, 2)};
    return pow(b, 2) - (4 * a * c) >= 0 and -b/a >= 0 and c/a >=
    ↪   0;
```

```
    // solve for t to see if the ray intersects with sphere, use
    ↪  discriminant to see if t exists
}
```

Using a nested `for` loop, the operations pipeline of obtaining a light ray for the pixel and determining if the ray intersects with the sphere can be applied for every pixel on the screen.

```
inline void displaySphere(const Custom::Camera& camera, const
↪  Sphere& sphere) {
    for (int pixelY {0}; pixelY < camera.screenHeight();
    ↪  ++pixelY) {
        for (int pixelX {0}; pixelX < camera.screenWidth();
        ↪  ++pixelX) {
            if(rayIntersectWithSphere(getPixelLine(pixelX,
            ↪  pixelY, camera), sphere)) {
                DrawPixel(pixelX, pixelY, sphere.color);
            }
        }
    }
}
```

A main function is used to tie in all the functions:

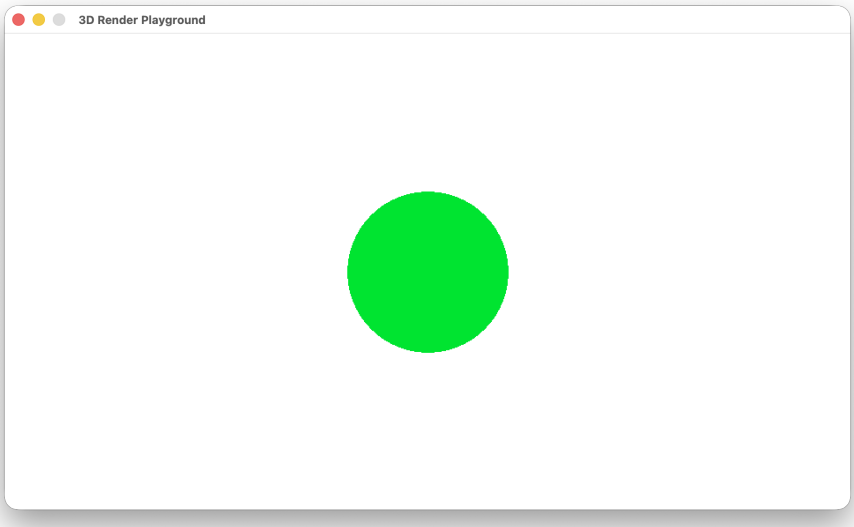
```
#include <raylib-cpp.hpp>
#include "camera.hpp"
#include "sphere.hpp"

int main() {
    Custom::Camera camera {};
    raylib::InitWindow(camera.screenWidth(),
    ↪  camera.screenHeight(), "3D Render Playground");
    Sphere sphere {{0, 0, 3}, 0.5, raylib::GREEN};
    DisableCursor();

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(raylib::WHITE);
        displaySphere(camera, sphere);
        EndDrawing();
    }

    return 0;
}
```

We have successfully ray traced a sphere to a 2D screen. Source code for this version can be found at `https://github.com/Wide0125/Raytracer-Version-1`.



3.3 Camera Movement

Our implementation allows us to render a sphere, but the calculations presume a viewport location at $(0, 0, 1)$. We will further refine our program to work for any viewport position.

The position of the viewport is always a fixed distance away from the camera. As such, two angles are used to define all possible positions of the viewport, azimuth and elevation.

By increasing or decreasing either of these angles, we can essentially 'turn' the camera to face any possible direction. Using sine and cosine values, we can then obtain the vector pointing from the camera to the center of the viewport. Afterwards, adjusting our functions to account for a dynamic camera-to-viewport-center vector makes facing different directions with the camera possible.

First, we create a `Degree` class object.

Source Code 4: degree.hpp

```
#include <cmath>
#include <numbers>

class Degree {
    public:
        Degree(double degree)
            : m_angle {degree}
        {}

        void add(double amount) {
            m_angle += amount;
            m_angle = std::fmod(m_angle, 360);
```

```
            if (m_angle < 0) {m_angle += 360;}
        }

        constexpr Degree plus(double amount) const {
            double ret {m_angle + amount};
            ret = std::fmod(m_angle, 360);
            if (ret < 0) {ret += 360;}
            return Degree{ret};
        }

        void set(double angle) {
            angle = std::fmod(angle, 360);
            if (angle < 0) {angle += 360;}
            m_angle = angle;
        }

        constexpr double get() const {
            return m_angle;
        }

        constexpr double radians() const {
            return m_angle * std::numbers::pi/180;
        }

        friend inline constexpr auto operator*(const Degree&,
        ↪  const auto);
        friend inline constexpr auto operator*(const auto, const
        ↪  Degree&);

        friend inline constexpr auto operator+(const Degree&,
        ↪  const Degree&);

    private:
        double m_angle {0};
};

inline constexpr auto operator*(const Degree& degree, const auto
↪  multiple) {return degree.m_angle * multiple;}
inline constexpr auto operator*(const auto multiple, const
↪  Degree& degree) {return degree.m_angle * multiple;}

inline constexpr auto operator+(const Degree& degree1, const
↪  Degree& degree2) {return degree1.plus(degree2.m_angle);}
inline constexpr Degree& operator+=(Degree& degree1, const double
↪  amount) {
    degree1.add(amount);
    return degree1;
}
inline constexpr Degree& operator+=(Degree& degree1, const
↪  Degree& degree2) {
```

```
    degree1.add(degree2.get());
    return degree1;
}
```

Then, we can add `xztheta` (azimuth) and `yztheta` (elevation) as new member variables of the camera class. Additionally, the vector denoting the direction from the camera to the viewport center is initialized as well.

```
Degree m_xzTheta {90};
Degree m_yzTheta {0};
Vector m_cameraToViewportCenter {
    m_viewportDistanceFromCamera * cos(m_yzTheta.radians()) *
    ↪  cos(m_xzTheta.radians()),
    m_viewportDistanceFromCamera * sin(m_yzTheta.radians()),
    m_viewportDistanceFromCamera * cos(m_yzTheta.radians()) *
    ↪  sin(m_xzTheta.radians())};
```

Two more member functions are added that enable changing the direction the camera is facing, updating the appropriate angle and vector variables.

```
void addAzimuth(double amount) {
    m_xzTheta += amount; // base of the right triangle with a
    ↪  hypotenuse of m_viewportDistanceFromCamera and angle
    ↪  yzTheta
    double yTriangleBase = m_viewportDistanceFromCamera *
    ↪  cos(m_yzTheta.radians());
    m_cameraToViewportCenter.x = yTriangleBase *
    ↪  cos(m_xzTheta.radians());
    m_cameraToViewportCenter.z = yTriangleBase *
    ↪  sin(m_xzTheta.radians());
}
void addElevation(double amount) {
    m_yzTheta += amount;

    if (m_yzTheta.get() > 90 and m_yzTheta.get() < 180)
    ↪  {m_yzTheta.set(90);}
    if (m_yzTheta.get() > 180 and m_yzTheta.get() < 270)
    ↪  {m_yzTheta.set(270);} // limit the range of the elevation
    ↪  (it doesn't make sense to be able to look up and then
    ↪  backwards)

    m_cameraToViewportCenter.y = m_viewportDistanceFromCamera *
    ↪  sin(m_yzTheta.radians());
    double yTriangleBase = m_viewportDistanceFromCamera *
    ↪  cos(m_yzTheta.radians()); // base of the right triangle
    ↪  with a hypotenuse of m_viewportDistanceFromCamera and
    ↪  angle yzTheta
    m_cameraToViewportCenter.x = yTriangleBase *
    ↪  cos(m_xzTheta.radians());
    m_cameraToViewportCenter.z = yTriangleBase *
    ↪  sin(m_xzTheta.radians());
}
```

Our `getPixelLine` function also needs updating to account for different viewport positions. Instead of offsetting the line vector from the camera-to-viewport-center vector by `DistanceFromCenter` in a fixed direction, now that the camera-to-viewport-center vector is dynamic, we must dynamically adjust the offset direction based on the direction of the camera-to-viewport-center vector. As the viewport is a rectangle, and the pixel positions map to a grid formation on the viewport, the offset vectors can be defined as illustrated in Figure 10. It can be noted that the `xVector` is orthogonal to the camera-to-viewport-center vector and the y axis, and the `yVector` is orthogonal to the `xVector` and the camera-to-viewport-center vector.

`getPixelLine` can be updated as such:

```
inline constexpr Line getPixelLine(int pixelX, int pixelY, const
↪  Custom::Camera& camera) {

    double xDistanceFromCenter
    ↪  {camera.viewportWidth()/camera.screenWidth() *
                               -(pixelX -
                               ↪  camera.screenWidth()/2)};
    double yDistanceFromCenter
    ↪  {camera.viewportHeight()/camera.screenHeight() *
                               -(pixelY -
                               ↪  camera.screenHeight()/2)};

    Vector cameraToViewportCenter
    ↪  {camera.cameraToViewportCenter()};
    Vector xVector {crossProduct(cameraToViewportCenter, {0, -1,
    ↪  0})};
    Vector yVector {crossProduct(cameraToViewportCenter,
    ↪  xVector)};

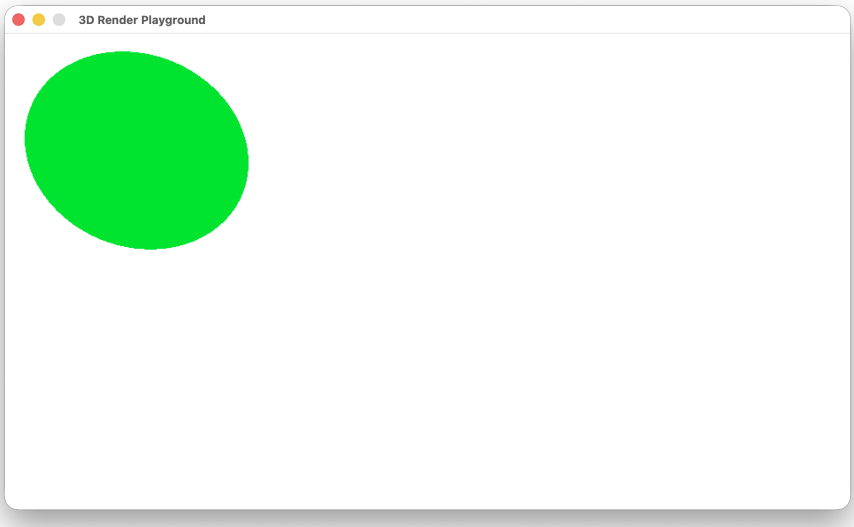
    double xMultiple
    ↪  {xDistanceFromCenter/sqrt(dotProduct(xVector, xVector))};
    double yMultiple
    ↪  {yDistanceFromCenter/sqrt(dotProduct(yVector, yVector))};

    return {camera.pos(), camera.cameraToViewportCenter() +
    ↪  xMultiple * xVector + yMultiple * yVector};
} // get vector from origin to pixel's equivalent location on
↪  viewport
```

Lastly, code is added to the `main` function that maps mouse movement to camera movement.

```
Vector2 mouseMovement {GetMouseDelta()};
camera.addAzimuth(mouseMovement.x/50);
camera.addElevation(-mouseMovement.y/50);
//down is +yDelta, up is -yDelta so mouse movement is inverted
```

As no more functions require further updating, our upgraded raytracer implementation is now complete. Source code for this version can be found at `https://github.com/Wide0125/Raytracer-Version-2`.



Although ray tracing is useful in allowing us to represent 3-dimensional objects in a mathematically accurate manner, it comes at the cost of scalability and performance. It is easy enough to ray trace mathematically simple objects such as spheres (as demonstrated); however, attempting to ray trace objects other than spheres proves to be more difficult. Although not impossible, as it is possible to approximate 3D shapes using 2D polygons and trace each polygon separately, the core obstacle lies in performance: it is simply too costly to compute every single ray-object interaction for every pixel. Therefore, a different method, one optimized for drawing polygons that is more performant at the cost of quality and accuracy, is needed for applications that require high framerates.

4 Rasterization

4.1 Polygon Projection

To implement our rasterizer, code from the raytracer will be reused where applicable. This includes the `Degree`, `Camera`, `Point`, and `Vector` classes.

Rasterization is a technique that involves approximating any 3D object into a collection of 2D polygons, commonly triangles. Triangles are commonly used as they are the simplest n-sided polygon, which every other polygon can be split into.

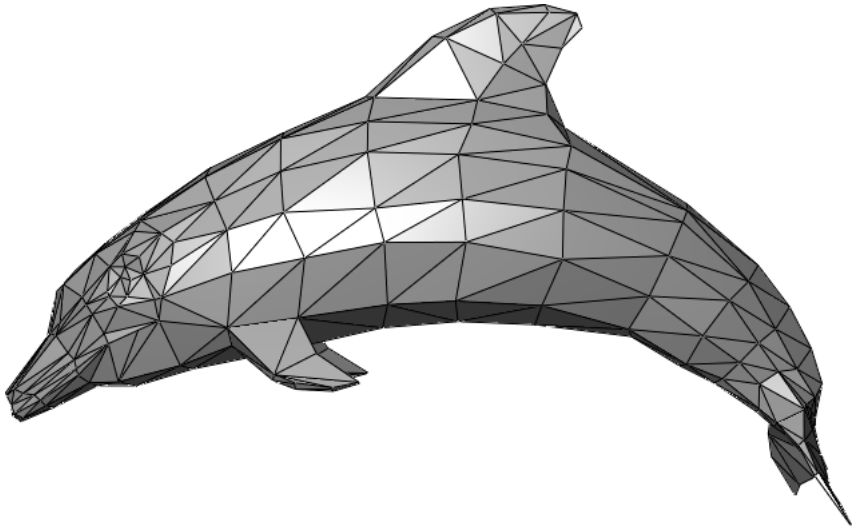


Figure 11: A 3D model approximated into triangles

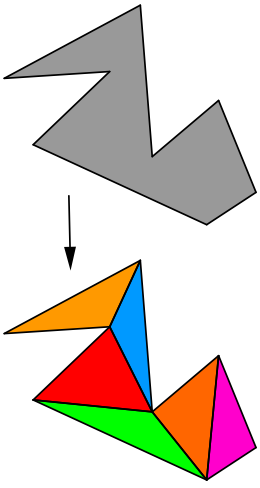


Figure 12: A demonstration of polygon triangulation

Approximating models into triangles is convenient, as all we have to figure

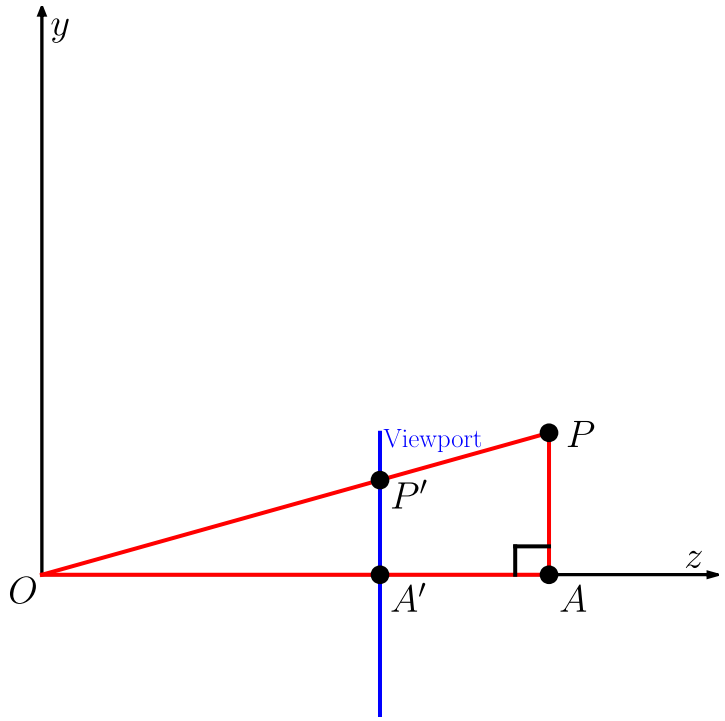


Figure 13: A side (yz plane) view of an arbitrary point P

out is how to properly display a triangle in 3D space onto a 2D plane, and repeat that process for every single triangle in the scene.

The next step is to determine a method to render a triangle suspended in 3D space onto a 2D plane. Focusing on a single point on the triangle P, and assuming a viewport center position of $(0, 0, 1)$ for simplicity, a side view of the point in relation to the viewport reveals 2 triangles, $\triangle OPA$ and $\triangle OP'A'$.

As with ray tracing, we take advantage of the fact that light travels in a straight line, meaning that P' is where we must draw P on the viewport. Thus, the task at hand is to calculate the position of P' of an arbitrary point P. It can be noted that point A is designated so that $\angle PAO$ is a right angle. Therefore, $\triangle OPA \sim \triangle OP'A'$, and as such, $\overline{A'P'} : \overline{AP} = \overline{OP'} : \overline{OP} = \overline{OA'} : \overline{OA}$. The coordinates of point P are given, point A is able to derived from point P by taking its z coordinate, and point A' and the z coordinate of P' are constants. Therfore, the only variable, the y coordinate of point P, is able to be derived:

$$P'_y = \frac{\overline{PA}}{\overline{OA}} \cdot \overline{OA'} = \frac{P_y}{P_z} \cdot 1$$

P'_x is able to be derived in a similar fashion, except this time from the view of the xy plane.

$$P'_x = \frac{P_x}{P_z} \cdot 1$$

We can convert the world space coordinates of P' into screen space coordinate with the formula below (where V_w and V_h refer to viewport width and height respectively, while S_w and S_h refer to screen width and height respectively):

$$P'_{cx} = -\frac{P'_x}{V_w} \cdot S_w + \frac{S_w}{2}$$

$$P'_{cy} = -\frac{P'_y}{V_h} \cdot S_h + \frac{S_h}{2}$$

Now that we have a method to obtain the P' coordinate of every point on the triangle P, all that is left is to fill in the rest of the triangle onto the screen. It can be noted that straight lines in 3D space remain straight even when projected onto a 2D surface; therefore, connecting the three points drawn onto the screen results in the correctly projected triangle.

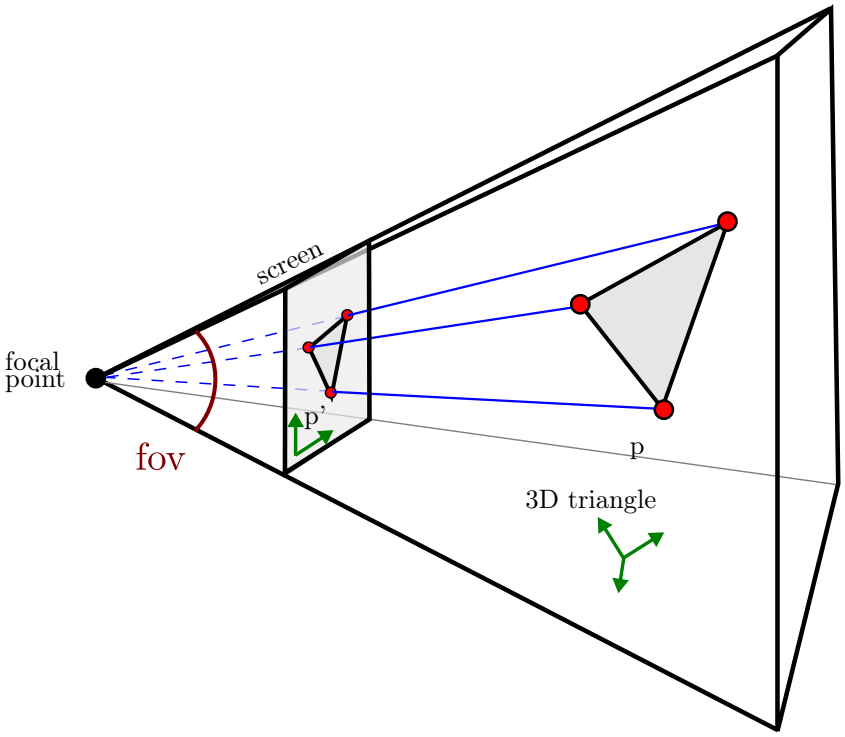


Figure 14: Projection of a 3D triangle onto a 2D viewport

To draw a triangle, we must first figure out how to connect two screen space coordinates. The equation of a line passing through two points A and B is as follows:

$$y - A_y = \frac{B_y - A_y}{B_x - A_x}(x - A_x)$$

Computer screens approximate images with pixels, so drawing a mathematically accurate line is not possible. However, an approximation is possible by rounding to the nearest integer. For every x coordinate of a pixel on the screen, the pixel's

y coordinate can be derived by plugging the value of x into our formula for the line. After that, the y value is rounded to the nearest integer to color in the pixel closest to the given x and y coordinates. Therfore, to draw a line between two pixel coordinates, a `for` loop that iterates over every integer x value between the two points is used. Every iteration, the y coordinate for that pixel is derived, and the pixel is colored in. An optimization can be made here by adding the slope of the line to the y coordinate every iteration instead of recalculating, as multiplication is simply repeated addition.

```
double slope {(y2 - y1) / static_cast<double>((x2 - x1))};
double y {static_cast<double>(x1)};

for (int x {x1}; x <= x2; ++x) {
    DrawPixel(x, y);
    y += slope;
}
```

However, a problem occurs when the slope of the line is greater than 1. Since we are drawing one pixel for every x coordinate, the algorithm results in disconnected lines (Figure 15). This happens because screen spaces are integer

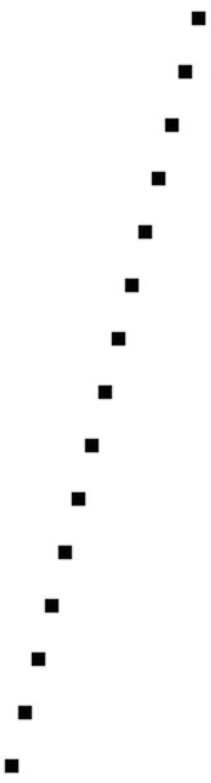


Figure 15: Disconnected lines

approximations of real numbers, and since the slope is greater than 1, for every 1 pixel difference in x, the y coordinate increases by more than 1, leaving gaps.

As such, for lines with a slope greater than 1, instead of drawing 1 pixel for every x coordinate, we need to draw 1 pixel for every y coordinate, filling in the gaps. To accomplish this, instead of iterating over every x value between the two points, we instead iterate over every y value between the two points and derive the x coordinate from the y values instead. This ensures that the drawn line remains connected.

This process can be generalized by designating an independent variable i, the variable we choose values for, and a dependent variable d, the variable whose values depend on i. In our `for` loop, we iterate over every integer i value and derive its appropriate d value. This is known as a linear interpolation function, a function that returns a line connecting two points. A linear interpolation function that returns an array of the dependent coordinates of a line connecting two points, $(i1, d1)$, $(i2, d2)$, in 2D space can be implemented as such:

```
std::vector<double> interpolate(int i1, int d1, int i2, int d2) {
    if (i1 == i2) {
        return {static_cast<double>(d1)};
    }

    if (i2 < i1) {
        std::swap(i1, i2);
        std::swap(d1, d2);
    }

    std::vector<double> retArray (i2 - i1 + 1);
    double slope {(d2 - d1) / static_cast<double>((i2 - i1))};
    double d {static_cast<double>(d1)};

    for (int i {i1}; i <= i2; ++i) {
        retArray[i - i1] = d;
        d += slope;
    }

    return retArray;
}
```

For a point (i, d) on the line, `retarray`$[i] = d$. For cases where the two points are the same, we simply return an array of a single point $d1$.

This generalization of the interpolating process is useful as it will serve as the foundation of our rasterizer.

```
void drawLine(int x1, int y1, int x2, int y2, raylib::Color color
↪   = BLACK) {
    std::vector<double> lineArray {};

    if (x2 - x1 > y2 - y1) {
        lineArray = interpolate(x1, y1, x2, y2);

        for (int x {x1}; x <= x2; ++x) {
            DrawPixel(x, lineArray[x - x1], color);
        }
```

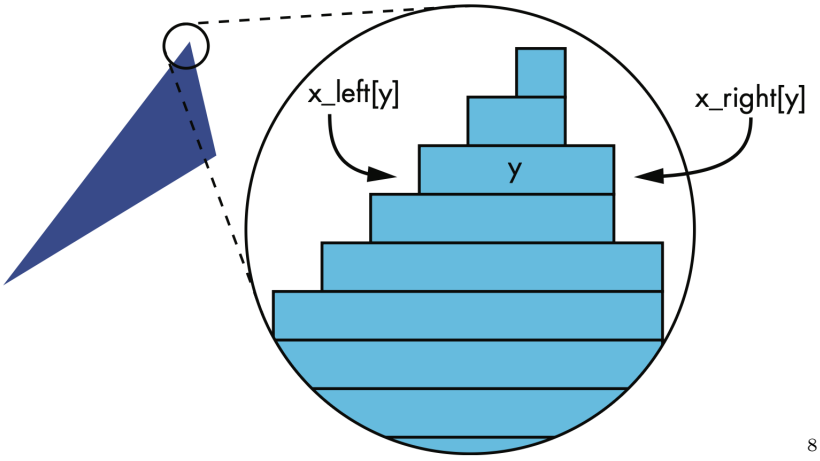
```
    }
    else {
        lineArray = interpolate(y1, x1, y2, x2);

        for (int y {y1}; y <= y2; ++y) {
            DrawPixel(lineArray[y - y1], y, color);
        }
    }
}
```

When the slope is less than 1, the linear interpolation function is called with x as the independent variable (i) and y as the dependent variable (d). When the slope is greater than or equal to 1, the function is instead called with y as the independent variable and x as the dependent variable.

With our `drawLine()` function and linear interpolation function, we can now draw 3D triangles onto a 2D screen. Now all that remains is to fill in the triangle.

We can go about accomplishing this in many ways, but one of the most straightforward methods is to imagine the process as painting in a horizontal strip of the triangle from the left edge all the way to the right edge, then working your way down and repeating until everything is painted in.



8

By sorting the 3 vertices of the triangle in terms of their y coordinates in descending order, we can designate two 'edges' that will serve as the left and right borders for our horizontal strips. One edge is the geometric edge between the highest point and the lowest point. The other edge is the combination of two geometric edges, the edge connecting the highest point with the middle point and the edge connecting the middle point with the lowest point. Which edge becomes the left edge and which edge becomes the right edge depends on the configuration of the points (Figure 16).

Our linear interpolation function comes in handy here, as it can be used to return arrays of every point along the necessary edges. For both edges, we must use the linear interpolation function with y as the independent variable, as we need to be able to color in every single y 'level' in the triangle. Once we obtain our left and right edges, we can iterate through both arrays simultaneously, and

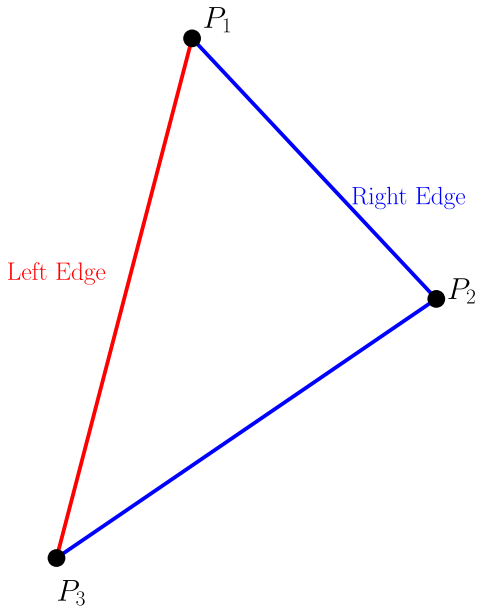


Figure 16

every operation, we color in the every pixel in between the left and right edges at the current y level.

```
void draw() const {
    std::vector<double> x123 {interpolate(m_p1.y, m_p1.x, m_p2.y,
    ↪  m_p2.x)};
    x123.pop_back();
    std::vector<double> x23 {interpolate(m_p2.y, m_p2.x, m_p3.y,
    ↪  m_p3.x)};
    x123.insert(x123.end(), x23.begin(), x23.end());
    std::vector<double> x31 {interpolate(m_p3.y, m_p3.x, m_p1.y,
    ↪  m_p1.x)};

    assert(x123.size() == x31.size());

    std::vector<double> x_left {};
    std::vector<double> x_right {};
    if (x31[x31.size() / 2] < x123[x123.size() / 2]) {
        x_left = x31;
        x_right = x123;
    }
    else {
        x_left = x123;
        x_right = x31;
    }
```

```
    for (int y {m_p1.y}; y <= m_p3.y; ++y) {
        for (int x {static_cast<int>(std::ceil(x_left[y -
        ↪  m_p1.y]))}; x <= x_right[y - m_p1.y]; ++x) {
            DrawPixel(x, y, m_color);
        }
    }
}
```

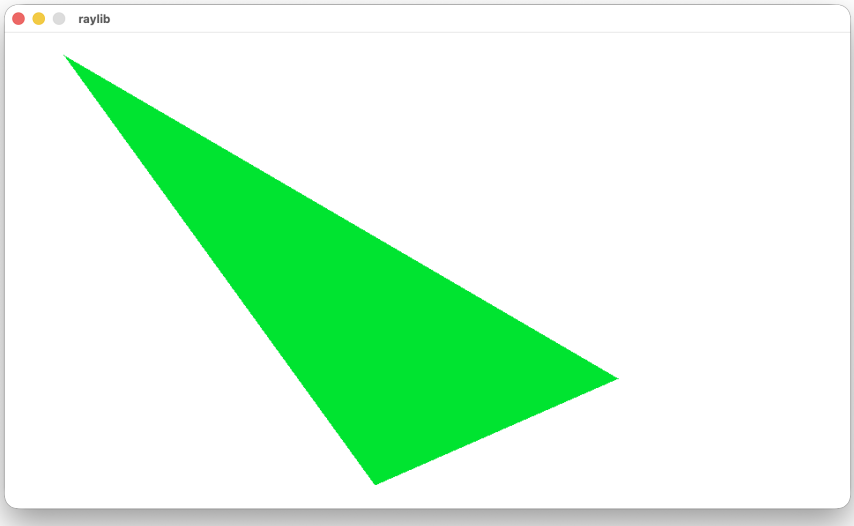


Figure 17: Our filled triangle result

4.2 Model Transformation

Let's say we would like to illustrate a scene, an ocean full of dolphins. In this case, we need to be able to render multiple dolphins in the scene, all with their own distinct positions, orientations, and sizes. This calls for the need to separately designate **models** and **model instances**. A model is an object containing all the information about its vertices and which ones connect to form triangles, and a model instance is an object that contains a model object, with additional member variables denoting its rotation, scale, and position. This also means that models exist in their own **model space** before being transformed into its world space coordinates depending on a model instance's member variables.

We first define a primitive version of our model and model instance class objects:

```
namespace Custom {
    struct Model {
        std::vector<Point3> vertices {};
```

```
        std::vector<std::array<int, 3>> triangles {{}};
        std::vector<raylib::Color> colors {triangles.size()};
        Point3 center {ORIGIN};
    };

    class ModelInstance {
        public:
            ModelInstance(Model model, Point3 position, double scale = 1)
                : m_model {model}, m_position {position}, m_scale {scale}
            {}

            void draw(Custom::Camera& camera) const;

            void addRoll(double amount);
            void addPitch(double amount);
            void addYaw(double amount);

        private:
            Model m_model {};
            Point3 m_position {0, 0, 0};
            double m_scale {1};
    };
}
```

A primitive `draw()` function can be implemented as such:

```
void Custom::ModelInstance::draw(Custom::Camera& camera) const {

    std::vector<ScreenPoint2> pointsOnScreen {};
    pointsOnScreen.reserve(m_model.vertices.size());
    for (auto a: m_model.vertices) {
        Vector deltas {m_position - m_model.center};
        double px {a.x/a.z};
        double py {a.y/a.z};

        int cx
        ↪  {static_cast<int>(std::round(px/camera.viewportWidth()
        ↪  * camera.screenWidth()))};
        int cy
        ↪  {static_cast<int>(std::round(py/camera.viewportHeight()
        ↪  * camera.screenHeight()))};
        pointsOnScreen.push_back({-cx + camera.screenWidth()/2,
        ↪  -cy + camera.screenHeight()/2});
    }
    int count {0};
    for (auto a: m_model.triangles) {
        Triangle{{pointsOnScreen[a[0]]}, {pointsOnScreen[a[1]]},
        ↪  {pointsOnScreen[a[2]]},
        ↪  m_model.colors[count]}.draw();
        ++count;
    }
}
```

Now, we must implement a `transform()` function that can apply the 3 transformations of the model instance: rotation, then scaling, then translation in that order. In order to achieve this, a closer examination of vector and space properties is required. For the sake of simplicity and ease of understanding, the following vector concepts will be explained in terms of 2 dimensions. These concepts can then be easily expanded to apply for 3 dimensions.

A vector is composed of scalar multiples of the unit vectors. A **unit vector** is a vector with length 1 that only travels along one axis. In 2 dimensions, the unit vectors are:

$$\hat{\imath} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \hat{\jmath} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Using the 2 unit vectors $\hat{\imath}$ and $\hat{\jmath}$, every possible vector can be expressed. Ex-

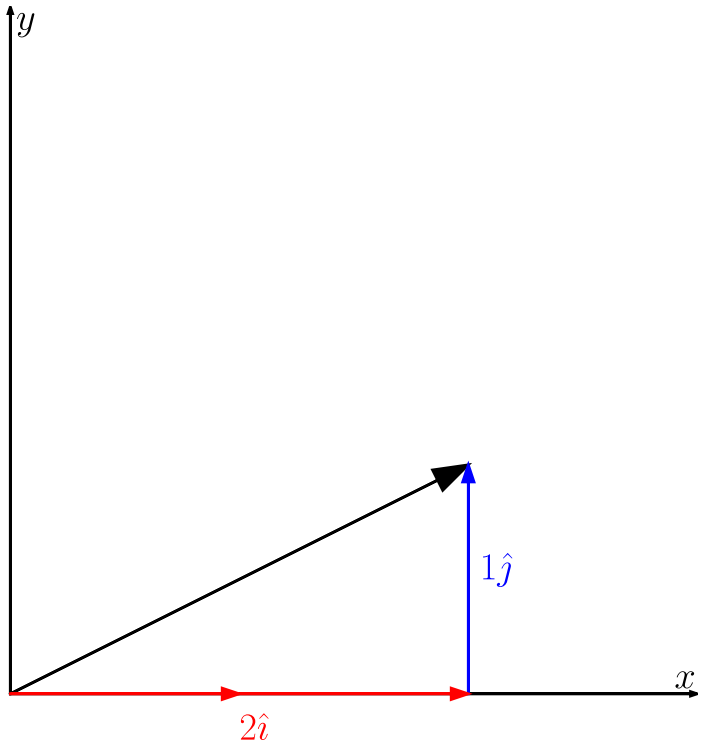


Figure 18: The vector $2\hat{\imath} + 1\hat{\jmath}$

tending the concept, we can say that space itself is defined with a combination of scalar multiples of the unit vectors serving as the basis; in other words, every possible point in space can be defined as a vector made up of scalar multiples of the unit vectors. This works because points and vectors are similar to the point of being nearly interchangeable. Any point can be swapped out for a vector going from the origin point to that point, and any computations involving that point also work with its equivalent vector; even the contents of their notations are conveniently similar. Going back to our use case, we can take advantage of the fact that transforming the unit vectors composing a vector also applies

that same transformation to the vector itself (Figure 19), then apply this for the points on our model.

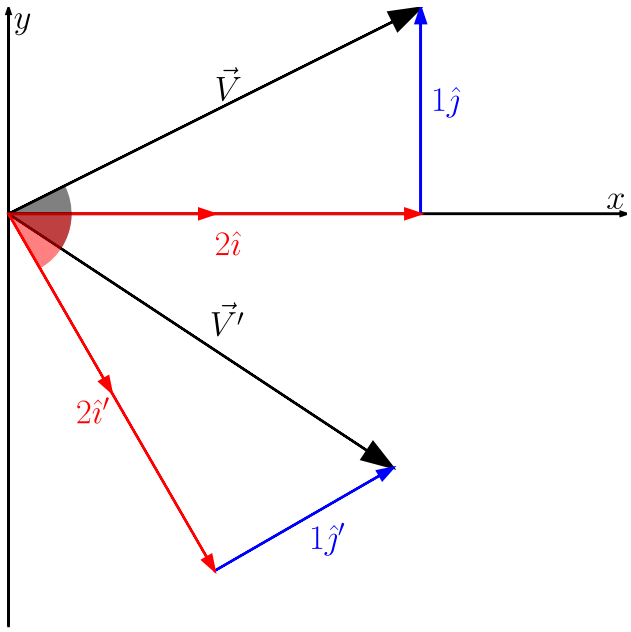


Figure 19: The angle between $\vec{V}$ and $\vec{V}\prime$ is equal to the angle between $\hat{\imath}$ and $\hat{\imath}\prime$

As such, we can store the information of the transformed unit vectors using a matrix. A **matrix** is a rectangular array of numbers, composed of rows and columns, useful for a variety of mathematical applications. In our case, we can use matrices to perform coordinate transformations.

We store the information of the transformed unit vectors into a matrix like so:

$$\hat{\imath}' = \begin{bmatrix} \hat{\imath}'_x \\ \hat{\imath}'_y \end{bmatrix} \hat{\jmath}' = \begin{bmatrix} \hat{\jmath}'_x \\ \hat{\jmath}'_y \end{bmatrix} \rightarrow M_t = \begin{bmatrix} \hat{\imath}'_x & \hat{\jmath}'_x \\ \hat{\imath}'_y & \hat{\jmath}'_y \end{bmatrix}$$

Where $\hat{\imath}'$ and $\hat{\jmath}'$ are the transformed unit vectors, and M_t is the transformation matrix. It can also be observed that matrices have similar notation to vectors, and this is not a coincidence: matrices can be thought of as an extension of vectors (they are simply storing multiple vectors), and vectors can be thought of as matrices with only 1 column.

Now that we have our transformation matrix, we can multiply it with a vector to apply the transformation. In simple terms, a vector notated as the set of its components $\vec{V} = \langle x, y \rangle$ tells us how much of each unit vector makes up the vector. For example, the vector $\langle 2, 1 \rangle$ is is the sum of $2\hat{\imath}$ and $1\hat{\jmath}$.

Our matrix stores the information of the transformed unit vectors, so we can obtain the transformed vector by multiplying the transformed unit vectors by the scalar components of the original vector. In other words, the transformed vector, instead of being composed of scalar multiples of the unit vectors $\hat{\imath}$ and

$\hat{\jmath}$, is now composed of scalar multiples of $\hat{\imath}'$ and $\hat{\jmath}'$ (Figure 19).

$$\vec{V} = \begin{bmatrix} a \\ b \end{bmatrix}$$

$$M_t = \begin{bmatrix} \hat{\imath}'_x & \hat{\jmath}'_x \\ \hat{\imath}'_y & \hat{\jmath}'_y \end{bmatrix}$$

$$\vec{V}' = M_t \times \vec{V} = \begin{bmatrix} a\hat{\imath}'_x + b\hat{\jmath}'_x \\ a\hat{\imath}'_y + b\hat{\jmath}'_y \end{bmatrix}$$

It is also possible to multiply a matrix by another matrix, resulting in a matrix that combines the two transformations:

$$\begin{bmatrix} a & c \\ b & d \end{bmatrix} \times \begin{bmatrix} e & g \\ f & h \end{bmatrix} = \begin{bmatrix} ea + cf & ag + ch \\ eb + df & bg + dh \end{bmatrix}$$

The resulting matrix describes the transformation of multiplying with the left matrix, then the right matrix. Note that transformations are not commutative ($M_a \times M_b \neq M_b \times M_a$).

Matrices are a much more complex topic and foundational to the field of linear algebra, but matrix concepts outside those relevant to our use case are outside the scope of this paper. As such, this paper will be adhering to only explaining and laying the groundwork for matrix operations useful for our rasterizer.

Now that we have a method to transform vectors around a point (in the above examples, the origin point O), we can begin to implement functions to transform our model. First, we define a `matrix` class:

```
namespace Custom {
    class Matrix {
        public:
            Matrix(size_t rows, size_t cols, const
            ↪  std::vector<double>& data = {})
                : m_rows {rows}, m_cols {cols}, m_data (rows *
                ↪  cols)
            {
                if (data.size() != 0) {m_data = data;}
            }

            double& operator()(size_t row, size_t col);
            double operator()(size_t row, size_t col) const;
            Matrix getColumn(size_t col) const;
            Matrix getRow(size_t row) const;
            void setColumn(size_t col, Matrix colMatrix);
            operator Point3() const;

            friend constexpr Matrix operator*(const Matrix&,
            ↪  const Matrix&);
            friend constexpr Matrix operator*(double scalar,
            ↪  const Matrix&);
            friend constexpr Matrix operator+(const Matrix&,
            ↪  const Matrix&);
```

```
        friend constexpr Matrix& operator+=(Matrix&, const
        ↪  Matrix&);


    private:
        size_t m_rows {1};
        size_t m_cols {1};
        std::vector<double> m_data {};
};

constexpr Matrix operator+(const Matrix& rMatrix, const
↪  Matrix& lMatrix) {
    assert(rMatrix.m_rows == lMatrix.m_rows and
    ↪  rMatrix.m_cols == lMatrix.m_cols);
    Matrix retMatrix {rMatrix.m_rows, rMatrix.m_cols};

    for (size_t row {0}; row < rMatrix.m_rows; ++row) {
        for (size_t col {0}; col < rMatrix.m_cols; ++col) {
            retMatrix(row, col) = rMatrix(row, col) +
            ↪  lMatrix(row, col);
        }
    }

    return retMatrix;
}

constexpr Matrix& operator+=(Matrix& lMatrix, const Matrix&
↪  rMatrix) {
    lMatrix = lMatrix + rMatrix;
    return lMatrix;
}


constexpr Matrix operator*(const Matrix& lMatrix, const
↪  Matrix& rMatrix) {
    assert(lMatrix.m_cols == rMatrix.m_rows);
    Matrix retMatrix {lMatrix.m_rows, rMatrix.m_cols};
    for (size_t rcol {0}; rcol < rMatrix.m_cols; ++rcol) {
        Matrix currColumn {lMatrix.m_rows, 1};
        for (size_t lrow {0}; lrow < lMatrix.m_rows; ++lrow)
        ↪  {
            currColumn += rMatrix(lrow, rcol) *
            ↪  lMatrix.getColumn(lrow);
        }
        retMatrix.setColumn(rcol, currColumn);
    }
    return retMatrix;
};
```

```
    constexpr Matrix operator*(double scalar, const Matrix&
    ↪  matrix) {
        Matrix retMatrix {matrix.m_rows, matrix.m_cols};
        for (size_t col {0}; col < matrix.m_cols; ++col) {
            for (size_t row {0}; row < matrix.m_rows; ++row) {
                retMatrix(row, col) = matrix(row, col) * scalar;
            }
        }
        return retMatrix;
    }
}
```

We can now add an `m_transform` matrix variable.

```
Custom::Matrix m_transform {m_scale * (m_rollTransform *
↪  m_pitchTransform * m_yawTransform)};
```

For convenience, all our `model` objects will have their center coordinates at the origin point of their model space, meaning any rotations will happen around the origin point of their model space. This streamlines the rotation process and allows us to avoid additional complexity in defining our rotation matrices.

To briefly summarize, the process of transforming a `model` object's model space coordinates into its `modelInstance` world space coordinates is as follows: we iterate through every single point in the model, applying a transformation matrix that is the rotation and scale matrices multiplied, then apply the translation by adding the value of its position coordinates.

This calls for the need to add a new matrix member variable `m_transform` in the `modelInstance` class.

```
Custom::Matrix m_transform {3, 3, {
    m_scale, 0, 0,
    0, m_scale, 0,
    0, 0, m_scale
}};
```

In 3 dimensions, 3 angles, known as Euler Angles, are needed to define every possible orientation of an object in 3D space. This is because there are 3 different combinations of the x, y and z axes that make a 2D plane. In 2 dimensions, as there is only 1 possible combination of the x and y axes, only 1 angle is needed.

Rotation matrices for rotations around each of the axes are as follows:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

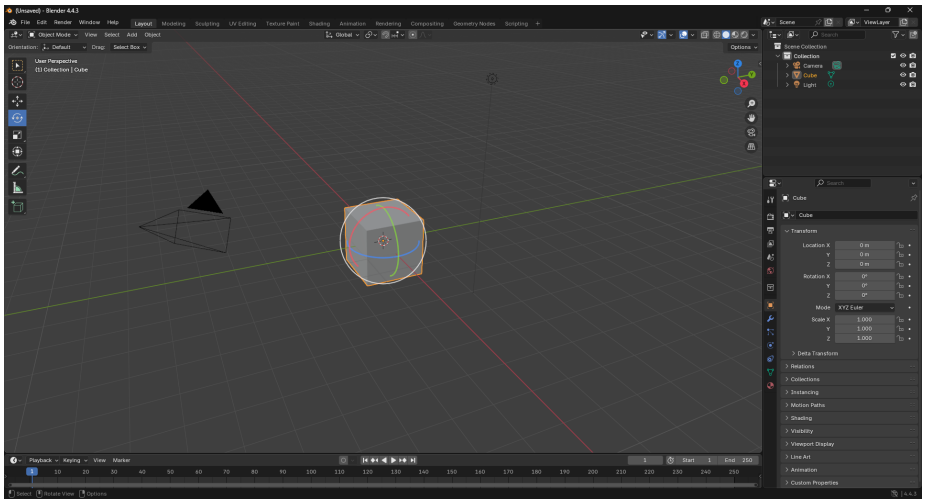


Figure 20: A preview of the rotation mechanism in Blender

As such, in order to be able to achieve every possible orientation in 3D space, an object must be able to rotate in 3 different planes, or in other words, around the 3 axes. Therefore, our rasterizer program will imitate the object rotation mechanism used in the open source 3D software Blender (Figure 20).

We can achieve this by implementing three new functions:

```
void Custom::ModelInstance::rotateX(double amount) {
    Custom::Matrix rollTransform = {3, 3, {
    1, 0, 0,
    0, std::cos(Degree{amount}.radians()),
    ↪  -std::sin(Degree{amount}.radians()),
    0, std::sin(Degree{amount}.radians()),
    ↪  std::cos(Degree{amount}.radians())}
    };
    m_transform = rollTransform * m_transform;
}
void Custom::ModelInstance::rotateY(double amount) {
    Custom::Matrix pitchTransform = {3, 3, {
    std::cos(Degree{amount}.radians()), 0,
    ↪  std::sin(Degree{amount}.radians()),
    0, 1, 0,
    -std::sin(Degree{amount}.radians()), 0,
    ↪  std::cos(Degree{amount}.radians())}
    };
    m_transform = pitchTransform * m_transform;
}
void Custom::ModelInstance::rotateZ(double amount) {
    Custom::Matrix yawTransform = {3, 3, {
    std::cos(Degree{amount}.radians()),
    ↪  -std::sin(Degree{amount}.radians()), 0,
    std::sin(Degree{amount}.radians()),
    ↪  std::cos(Degree{amount}.radians()), 0,
```

```
    0, 0, 1}
    };
    m_transform = yawTransform * m_transform;
}
```

We then create a `transform` function that takes in a point as input and multiplies it by the `m_transform` matrix (treating the point as a vector from the origin to that point), and then also applies the translation afterwards.

```
Point3& Custom::ModelInstance::transform(Point3& modelPoint)
↪   const {
    modelPoint = m_transform * modelPoint;

    modelPoint.x += m_position.x - m_model.center.x;
    modelPoint.y += m_position.y - m_model.center.y;
    modelPoint.z += m_position.z - m_model.center.z;

    return modelPoint;
}
```

We then add a line of code to our `draw()` function inside the first `for` loop to transform every single point in the model.

```
a = transform(a);
```

Lastly, just like with the raytracer, we need to implement camera movement. It should be noted that when the camera's viewport rotates by a certain angle, from the perspective of the camera, the object rotates around the camera in the opposite direction by that angle amount instead. Imitating this, instead of moving the camera in the world space like with the raytracer, we instead define a camera space where the camera is always at $(0, 0, 0)$ and the viewport is always centered at $(0, 0, 1)$. After that, we calculate how much the camera's coordinates and the viewport's orientation in relation to the camera in the world space deviate from their configurations in camera space. Then, we apply the reverse of that deviation onto every object before placing them into our camera space to be displayed on the screen.

Although this sounds complicated, in reality, this is as simple as adding a single line of code. If we assume the camera never moves from the origin point for the sake of simplicity, we can add the following line in our `transform()` function after the rotation and translation happen to ensure that camera movement is applied correctly.

```
modelPoint = camera.m_azimuthTransform *
↪   (camera.m_elevationTransform * modelPoint);
```

It should be noted that the object is rotated by the elevation angle before the azimuth angle, due to how our viewport angles are configured.

With our rasterizer, it is possible to display objects with more complex geometry (Figure 21).

Source code for the rasterizer is available at `https://github.com/Wide0125/rasterizer`.

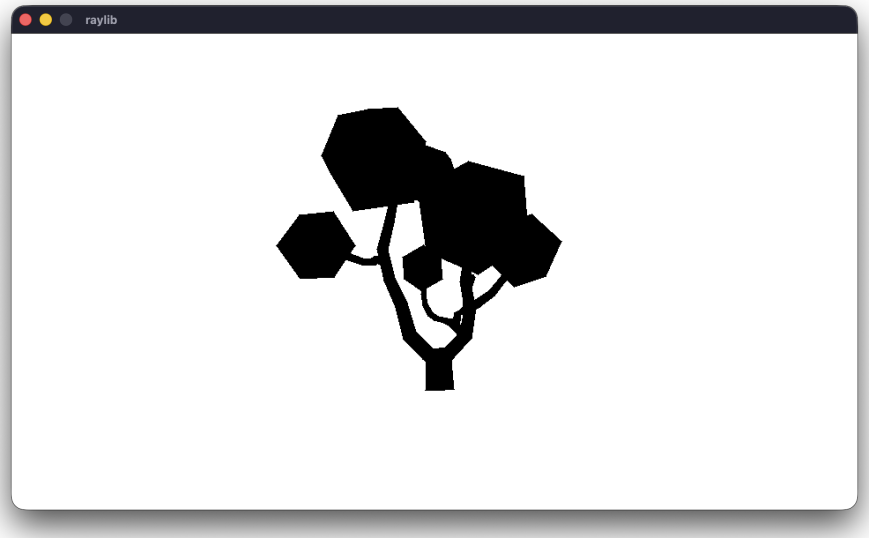


Figure 21: A model of a tree rendered with our rasterizer

5 Conclusion

Ray tracing and rasterization are two different approaches to illustrating 3D scenes onto a 2D plane. In ray tracing, a more closer-to-reality approach is used, tracing the path of light rays that hit the camera in reverse and using mathematical definitions of rays and objects to determine the property of the projected light. In rasterization, objects are approximated into triangles, optimizing the process of drawing a scene by reducing the number of computations required; instead of having to trace a ray for every pixel, we instead compute for every point of a model instead, and take advantage of the property of projected triangles to connect the points into a polygon. This approach drastically improves the performance of the rendering process.

A ray tracer is proficient at rendering physically accurate objects in a simple and intuitive process. It is also superior to rasterizers when it comes to lighting, as it is modeled as closely as possible after real world light behaviors. However, a ray tracer falls short when it comes to performance; having to perform computationally intensive calculations many times every frame, ray tracing drastically decreases the number of frames able to computed each second. However, as hardware continually improves in performance capabilities, this drawback is expected to become less of a problem.

A rasterizer is proficient at using clever geometry and approximation methods to simplify the process of rendering objects computationally. This means that a rasterizer can quite easily render a scene with thousands of triangles. However, a rasterizer falls short in terms of accuracy, as it is only an approximation of the real world. It can also be very complicated to implement, with a rasterizer relying on abstract mathematical concepts rather than ones much more closely based in real world physics.

Overall, both techniques have their use cases. Although today, the wide variety of ready-made graphics libraries and 3D software make knowledge of the underlying mechanics behind these techniques seemingly obsolete, knowledge of the fundamentals is evidently still crucial in further improving hardware and computer visuals. This paper seeks to make such knowledge more accessible to a general audience, giving readers the tools and skills they need in order to further improve the world of 3D computer graphics.

References

Eldridge, Stephen. *Cartesian coordinates*. 2023. URL: https://www.britannica.com/science/Cartesian-coordinates (visited on 10/08/2025).

Encyclopedia Britannica, The Editors of. *line*. 2009. URL: https://www.britannica.com/science/line-mathematics (visited on 10/08/2025).

— *vector*. 2004. URL: https://www.britannica.com/science/vector-mathematics (visited on 10/08/2025).

Gambetta, Gabriel. *Computer Graphics From Scratch*. William Pollock, 2021. ISBN: 978-1-7185-0077-8.

— *Computer Graphics From Scratch*. 2021.

Loach, Rob. *raylib-cpp*. Version 5.5.0. URL: https://github.com/robloach/raylib-cpp.

Santamaria, Ramon. *Raylib*. Version 5.5. URL: https://www.raylib.com/.